K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 2

Unit Title: Linear Data Structures

Assignment Title: Linear Data Structures

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Shreya Verma

Roll Number: 2501730173

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

1. Introduction

Linear Data Structures store data elements in a sequential manner where each element is connected to its previous and next element. Common examples include arrays, linked lists, stacks, and queues.

In this project, we implemented a Linear Data Structures Toolkit (LDST) using Python. The aim was to understand the internal working of these data structures by building them from scratch and applying them to solve a real-world problem.

2. Dynamic Array

Concept

A Dynamic Array is a resizable array that automatically increases its capacity when it becomes full. Unlike static arrays, it can grow dynamically during runtime.

Implementation Details

- The array starts with an initial capacity.
- When the array becomes full:
  - A new array with **double capacity** is created.
  - All elements are copied into the new array.
- Elements are then appended normally.

Operations Implemented

- `append(x)`
- `pop()`
- Display array

Amortized Time Complexity of Append

- Most `append()` operations take **O(1)** time.
- When resizing occurs:
  - Copying elements takes **O(n)** time.

However, resizing does not happen frequently.

Conclusion

Even though some operations are expensive, the **average (amortized) time complexity of append is O(1)**.

3. Singly Linked List

Concept

A Singly Linked List consists of nodes where each node contains:

- Data
- A pointer to the next node

Operations Implemented

- Insert at beginning
- Insert at end
- Delete by value
- Traverse list

Advantages

- Dynamic size
- Efficient insertion and deletion
- No shifting of elements required

4. Doubly Linked List

Concept

Each node contains:

- Data
- Pointer to next node
- Pointer to previous node

Operations Implemented

- Insert after a given node
- Delete at a specific position

Advantages

- Traversal in both directions
- Easier deletion compared to singly linked list

5. Stack Implementation (Using Linked List)

Concept

Stack follows the **LIFO (Last In First Out)** principle.

Operations

- `push(x)` → Insert element
- `pop()` → Remove top element
- `peek()` → View top element

Implementation Detail

- Operations are performed at the **head of the linked list** to achieve constant time complexity.

Time Complexity

Operation	Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)

6. Queue Implementation (Using Linked List)

Concept

Queue follows the **FIFO (First In First Out)** principle.

Operations

- `enqueue(x)` → Insert element
- `dequeue()` → Remove front element
- `front()` → View front element

Implementation Detail

- Uses both **head and tail pointers** for efficient operations.

Time Complexity

Operation	Complexity
Enqueue	O(1)
Dequeue	O(1)
Front	O(1)

7. Application: Balanced Parentheses Checker

Problem Statement

Check whether a given expression has balanced parentheses.

Approach

- Use a **Stack** data structure:
  - Push opening brackets (`(`, `{`, `[`)
  - Pop when closing brackets appear
  - Match pairs correctly

Conditions for Balanced Expression

- Every opening bracket must have a closing bracket
- Brackets must be in correct order

Test Cases

Input	Output
([])	Balanced
([)]	Not Balanced
(((	Not Balanced
""	Balanced

8. Learning Outcomes

From this project, I learned:

- How dynamic arrays resize internally
- Differences between arrays and linked lists
- Implementation of stack and queue using linked lists
- Importance of time complexity in algorithm design
- Practical application of stacks in solving problems

9. Conclusion

This project provided a deep understanding of linear data structures and their real-world applications. Implementing these structures from scratch enhanced problem-solving skills and clarified how data is organized and manipulated efficiently.

10. References

- Class Notes
- Python Official Documentation
- Online resources (for conceptual understanding)